

On-Demand PKRU Updates for Memory Protection Keys

UNIST CSE Course Project Report, December 2025

Joanita Nakityo

Ulsan National Institute of Science and Technology
(UNIST)

Department of Computer Science and Engineering
Ulsan, South Korea

Hyeonseok Oh

Ulsan National Institute of Science and Technology
(UNIST)

Department of Computer Science and Engineering
Ulsan, South Korea

1 Introduction

Modern applications increasingly execute untrusted components within the same address space, including plugin systems, multi-tenant services, and serverless runtimes. These environments require memory-isolation mechanisms that are both lightweight and predictable. Intel Memory Protection Keys (MPK) provides such a mechanism: each page-table entry carries a 4-bit protection key, and threads can update access rights using the `WRPKRU` instruction without modifying page tables or triggering TLB shootdowns. This enables permission switching in tens of cycles, avoiding the high overheads of `mprotect()`, software fault isolation, or address-space switching.

Despite these advantages, MPK remains difficult to use effectively in practice. Prior work primarily focuses on extending MPK’s scalability or semantics rather than preserving its deterministic performance. `LIBMPK` [3] virtualizes hardware keys but incurs page-table relabeling and global TLB shootdowns during eviction. `EPK` [1] improves virtualization efficiency yet still requires PTE updates for key reassignment. `VDom` [4] scales domains using multiple address spaces, but domain switches cost hundreds of cycles, while `Moat` [2] relies on page-table permissions for most domains.

A key observation is that *none of these systems preserve MPK’s deterministic, WRPKRU-only fast path*. Whether through virtualization, address-space scaling, or hybrid isolation, existing approaches introduce slow-path events such as PTE relabeling, eviction, or TLB shootdowns. These operations cause unpredictable latency spikes, undermining MPK’s core benefit for performance-critical applications.

This report takes a different perspective. Rather than scaling MPK to support more domains, we aim to *preserve MPK’s original performance model*. We argue that MPK is most compelling when domain switching can be guaranteed to occur without page-table modifications. To this end, we present *On-demand PKRU Updates*, a kernel-level mechanism that separates isolation domains into *hot* and *cold* groups, ensuring that hot domains always switch permissions using `WRPKRU` alone. The remainder of this report describes the motivation, design, evaluation, and limitations of our approach.

2 Motivation

Although Intel MPK provides nanosecond-scale permission switching, applying it in real systems is non-trivial due to its fundamental limit of 16 hardware protection keys. When applications require more domains than the available keys—which is common in plugin-based systems, multi-tenant runtimes, storage engines, or large multi-threaded libraries—the operating system or a virtualization layer must reassign keys. Such reassignment requires page-table relabeling, cross-core TLB shootdowns, and global synchronization, destroying MPK’s deterministic fast-path semantics.

Prior work attempts to scale MPK beyond its 16-key restriction, but all existing approaches sacrifice latency predictability. `LIBMPK` [3] introduces software-level key virtualization and fixes semantic issues such as protection-key-use-after-free and key aliasing. However, when the key cache is full, `libmpk` evicts a domain by relabeling its PTEs and flushing TLB entries system-wide, producing large tail-latency spikes.

`EPK` [1] further optimizes MPK virtualization by maintaining the same protection-key layout across up to 512 EPTs and switching between them with `VMFUNC`. This allows hundreds of virtual domains while avoiding per-switch PTE relabeling. Although `EPK` significantly reduces overhead compared to `libmpk` and often achieves performance close to a baseline without isolation, EPT switching is still fundamentally heavier than `WRPKRU`-only transitions and cannot match MPK’s native fast-path determinism.

`VDom` [4] eliminates the key-cardinality limit by using multiple address spaces tagged with ASIDs/PCIDs, enabling practically unlimited virtual domains. While average overhead remains low, VDS switches and virtual-domain eviction events cost hundreds to thousands of cycles and require partial TLB invalidation, far exceeding the tens-of-cycles cost of `WRPKRU`.

`Moat` [2] applies MPK’s kernel counterpart (PKS) to isolate kernel and BPF contexts. It uses PKS to protect the hot boundary between kernel and BPF, while relying on separate address spaces with PCID to isolate BPF programs among themselves. This two-layer design avoids most TLB flushes, but still incurs address-space switching overhead for inter-BPF transitions.

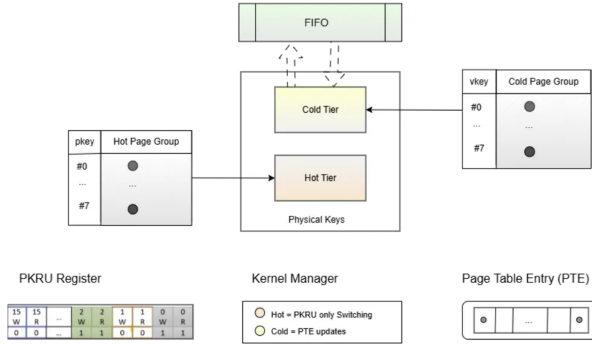


Figure 1. On-demand PKRU Updates with hot-cold domain tiering. Hot domains execute entirely on the WRPKRU-only fast path, whereas cold domains incur PTE updates only when accessed.

Across virtualization-based (libmpk, EPK) and address-space-based (VDom, Moat) designs, a common pattern emerges: scalability is improved, but MPK’s deterministic WRPKRU-only fast path is sacrificed. For latency-critical systems—serverless runtimes, storage engines, in-process multi-tenant execution—occasional eviction, EPT switching, or address-space switching introduces unacceptable jitter.

This leads to a different question: *can we preserve MPK’s deterministic switching semantics, rather than extending MPK through virtualization or address-space scaling?* This work explores an OS-level mechanism that protects the fast path by design.

3 Proposed Idea

We propose *On-demand PKRU Updates*, a kernel-level mechanism that strictly preserves MPK’s WRPKRU-only fast path for frequently accessed domains. As illustrated in Figure 1, our design avoids key virtualization and multiple address spaces, and instead restructures the operating system so that page-table modifications, key reassignment, and TLB shutdowns are confined to a controlled slow path, never occurring on the latency-critical fast path.

3.1 Hot-Cold Domain Tiering

Most applications exhibit heavy-tailed access distributions, with a small number of domains dominating execution. We exploit this by permanently binding hardware keys to *hot* domains. Once assigned, a hot domain’s mapping never changes, guaranteeing that all permission switches among hot domains remain WRPKRU-only. No eviction, no PTE relabeling, and no TLB shutdowns can affect these domains.

Cold domains, by contrast, are placed in a small FIFO pool of reusable protection-key slots. These domains may undergo PTE relabeling when they are first accessed or when their key slot is reassigned. Because cold domains are infrequently

accessed, this slow-path cost is amortized and isolated from hot-domain execution.

This tiered structure differs fundamentally from libmpk and EPK, whose eviction policies may affect frequently accessed domains, and from VDom or Moat, where domain switching inherently depends on heavier address-space mechanisms. Our design ensures that hot domains remain immune to all virtualization overhead.

3.2 Fault-driven Promotion and Controlled Slow Path

Cold-domain accesses trigger a SEGV_PKUERR protection-key fault. This hardware-delivered boundary provides a natural entry point into the slow path. Upon fault, the kernel identifies the domain, assigns or reuses a cold-slot key, and performs the necessary PTE relabeling. If the cold pool is full, the oldest cold domain is evicted using a FIFO policy.

Crucially, hot domains never fault and never participate in eviction. All slow-path costs are isolated to cold domains, guaranteeing that hot-domain switching latency remains constant.

3.3 Fast-path Invariants

Our design enforces the following invariants:

- **Invariant 1:** Switching among hot domains is always WRPKRU-only.
- **Invariant 2:** Hot domains never experience PTE modifications or TLB shutdowns.
- **Invariant 3:** Cold-domain faults encapsulate all expensive operations.
- **Invariant 4:** The number of hot domains can be tuned for application characteristics.

These invariants provide predictable memory isolation behavior with strictly bounded latency. Unlike EPK, libmpk, or VDom, our design does not sacrifice MPK’s hardware semantics for scalability; instead, it isolates slow-path work while preserving MPK’s deterministic performance where it matters.

4 Evaluation

We evaluate On-demand PKRU Updates to answer three questions: (1) How do hot and cold memory accesses differ in latency? (2) How does our design compare against existing MPK-based mechanisms? (3) Does hot-cold separation eliminate runtime overhead on the fast path?

4.1 Microbenchmark Setup

We use microbenchmarks to measure memory access latency across different execution paths. Each benchmark repeatedly accesses protected pages and records access latency. We report P50, P95, and P99 latencies to capture both average and tail behavior.

We evaluate three paths:

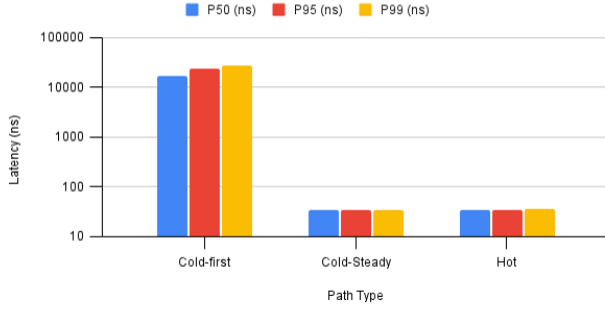


Figure 2. Access latency comparison across Hot, Cold-first, and Cold-steady paths. Cold-first accesses incur higher latency due to page faults and protection setup, while Hot and Cold-steady paths exhibit WRPKRU-only fast-path behavior.

- **Cold-first:** the first access to a cold domain, triggering a protection-key fault.
- **Cold-steady:** subsequent accesses after the domain is mapped.
- **Hot:** accesses to domains permanently assigned to hardware protection keys.

4.2 Hot and Cold Path Latency

Figure 2 compares access latency across the three paths. The **Cold-first path shows significantly higher latency**, as the initial access triggers a page fault, kernel mapping, and protection-key assignment, incurring user–kernel transitions.

In contrast, the **Hot path maintains consistently low latency**, since permission changes are handled entirely in user space via a single WRPKRU instruction. Notably, the **Cold-steady path closely matches the Hot path**, indicating that slow-path costs are paid only once and fully amortized.

4.3 Comparison with Existing Mechanisms

Figure 3 compares our design against a baseline system and prior approaches. The **Baseline (No MPK)** represents direct memory access without permission control.

As shown, **libmpk substantially outperforms mprotect** by avoiding frequent page-table permission updates, yet still incurs overhead due to key eviction and relabeling. Our **Hot/Cold tiered design further reduces overhead** by ensuring that frequently accessed regions execute entirely on the WRPKRU-only fast path. Consequently, our system achieves approximately **20× lower overhead than libmpk** and over **1700× improvement compared to mprotect**.

4.4 End-to-End Overhead and Latency Distribution

Figure 4 summarizes overall performance. The left plot shows runtime overhead relative to the baseline: both the **Hot** and **Cold-steady** paths incur near-zero overhead, occasionally exhibiting slight negative overhead due to measurement noise.

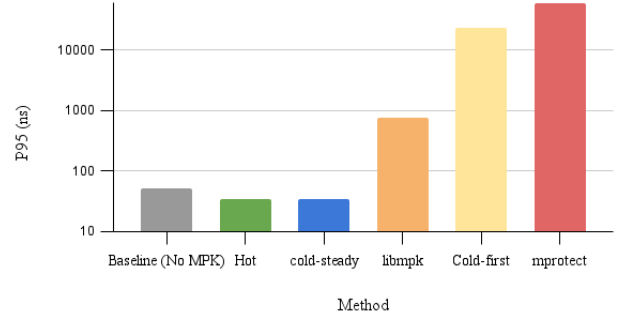


Figure 3. P95 latency comparison against Baseline, mprotect, and libmpk. Our Hot/Cold design eliminates page-table updates and system calls on the fast path.

The right plot presents the latency CDF. The **Cold-first distribution is shifted right**, reflecting one-time page fault costs, while **Hot and Cold-steady curves overlap tightly**, demonstrating low and predictable latency.

Overall, these results confirm that isolating expensive operations to a controlled slow path preserves fast-path performance while maintaining strong memory protection.

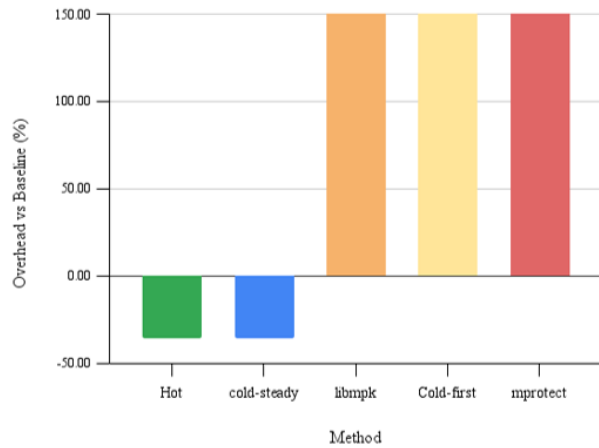
5 Limitations

While On-demand PKRU Updates restores MPK’s deterministic fast path, it has several limitations.

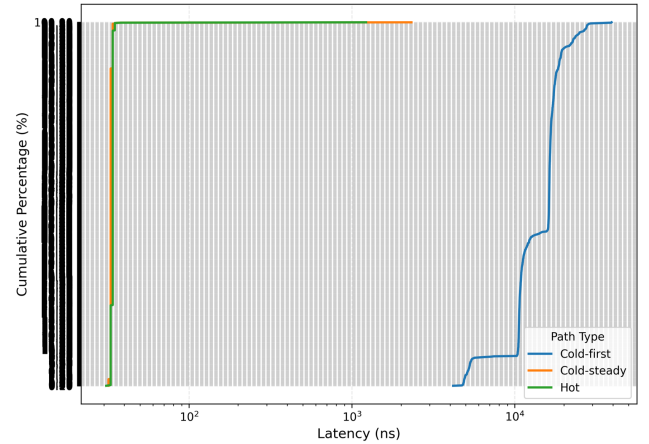
First, our current prototype is implemented in user space, requiring explicit management of hot–cold transitions. Promotion and demotion are handled manually in the code, for example by checking whether a page group exceeds a fixed access count. In a kernel-managed implementation, this process could be automated: the kernel could directly monitor protection-key faults and page access statistics using existing page-tracking mechanisms, and promote or demote pages transparently based on observed behavior. Our prototype therefore reflects a functional proof of concept rather than a fully integrated kernel solution.

Second, the prototype relies on a static promotion threshold, currently set to 100 accesses. Real workloads often exhibit diverse and time-varying access patterns, for which a fixed threshold may be suboptimal. Adaptive policies that incorporate recent access frequency, time decay, or workload-specific signals could improve performance and reduce unnecessary promotions. Exploring such dynamic thresholding mechanisms remains future work.

Third, the design assumes that a small number of domains dominate memory accesses. This assumption holds for applications with skewed access patterns, such as storage engines or serverless runtimes, but may not apply to workloads with uniformly distributed or highly bursty domain usage. In such cases, cold-domain faults may occur more frequently, increasing PTE relabeling events and reducing overall benefit,



(a) Runtime overhead relative to baseline (P95 latency).



(b) CDF of access latency for Cold-first, Cold-steady, and Hot paths.

Figure 4. Overall performance summary.

potentially approaching the behavior of MPK virtualization schemes such as libmpk or EPK.

Fourth, cold-domain eviction still requires page-table re-labeling and may trigger cross-core TLB shutdowns. Although these operations are isolated from the hot path, they are system-wide events and may introduce indirect overhead in highly parallel workloads, a limitation shared with designs such as VDom and libmpk.

Finally, our approach does not aim to support arbitrarily many isolation domains. Unlike EPK or VDom, which explicitly scale domain count through virtualization or multiple address spaces, On-demand PKRU Updates prioritizes preserving fast-path performance for a configurable subset of domains. Supporting both strict scalability and deterministic switching remains future work.

Despite these limitations, the framework effectively preserves MPK’s fast-path semantics for applications with skewed access patterns and stringent latency requirements.

6 Conclusion

Intel MPK provides uniquely lightweight, predictable permission switching, but existing virtualization and domain-scaling mechanisms fail to preserve its fast-path performance. This report presented On-demand PKRU Updates, a kernel-level approach that separates isolation domains into hot and cold tiers to ensure that hot-domain transitions remain strictly WRPKRU-only. By shifting all slow-path work to controlled cold-domain operations, our design restores MPK’s intended performance model while remaining compatible with scalability-focused systems such as LIBMPK, EPK, VDom, and Moat. The result is a practical, latency-predictable memory isolation mechanism suitable for performance-critical and multi-tenant applications.

References

- [1] Jinyu Gu, Hao Li, Wentai Li, Yubin Xia, and Haibo Chen. 2022. {EPK}: Scalable and efficient memory protection keys. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 609–624.
- [2] Hongyi Lu, Shuai Wang, Yechang Wu, Wanning He, and Fengwei Zhang. 2024. {MOAT}: Towards Safe {BPF} Kernel Extension. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1153–1170.
- [3] Soyeon Park, Sangho Lee, Wen Xu, Hyungon Moon, and Taesoo Kim. 2019. libmpk: Software abstraction for intel memory protection keys (intel {MPK}). In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 241–254.
- [4] Ziqi Yuan, Siyu Hong, Rui Chang, Yajin Zhou, Wenbo Shen, and Kui Ren. [n. d.]. Vdom: Fast and unlimited virtual domains on multiple architectures. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 905–919.